

Slovenská technická univerzita v Bratislave
Fakulta informatiky a informačných technológií
Communication application using the UDP protocol

Komunikácia s využitím UDP protokolu

Obsah

Úvod.....	3
Štruktúra hlavičiek protokolu	4
1.1 Opis hlavičky v bajtovej reprezentácii.....	4
1.2 Štruktúra hlavičky vlastného protokolu	5
1.3 Opis použitých polí v hlavičke.	5
Metód na kontrolu integrity prenesenej správy	6
2.1 Úvod do metódy kontrolu integrity prenesenej správy	6
2.2 Metoda na tvorbu chýb počas prenosu dát.....	6
2.3 Ako funguje CRC16?.....	6
2.4 Výpočet CRC16.....	6
2.5 Zvolený CRC (CRC-16-CCITT) a implementácia	8
Metóda na zabezpečenie spoľahlivého prenosu dát(ARQ)	8
3.1 Metódy na zabezpečenie spoľahlivého prenosu dát.....	8
3.2 SR metóda.....	9
3.3 Výber veľkosti okna (N).....	9
3.4 Príklad okna	9
3.5 Efektívnosť algoritmu SR.....	10
Metóda na udržanie spojenia – Keep-Alive.....	11
4.1 Úvod do metódy Keep-Alive	11
4.2 Ako funguje metóda na udržanie spojenia, a jej parametre	11
4.3 Ako sa nastavuje pripojenie.....	11
4.4 Ako sa ukončuje pripojenie.....	12
Popis časti programu.....	13
4.1 Zatáčok programu.....	13
4.2 Konfigurácia v súbore	14
4.2 Iné vlastnosti programu.....	15
Testovacie príklady so Wireshark.....	17
Prenos bez chýb:.....	17
Prenos s chybami:	18
Prenos keep-alive:	19
Diagramy opisujúce predpokladané správanie uzla/uzlov	20
REFERENCE :	21

Úvod

Úlohou tohto článku je dôkladne popísať navrhnutý protokol na základe protokolu UDP (User Datagram Protocol), ktorý bude použitý na tvorbu P2P aplikácie v aplikačnej vrstve sieťového modelu. Aplikácia umožní komunikáciu dvoch účastníkov(uzlov) v lokálnej Ethernet sieti, vrátane výmeny textu a prenosu ľubovoľných súborov medzi počítačmi (uzlami). Oba uzly budú fungovať súčasne ako prijímač aj odosielateľ.

Požiadavky na prototyp

Vytvorený prototyp protokolu spĺňa požiadavky, ako napríklad:

- byť schopný na nadviazanie spojenia medzi dvoma stranami a dohodnutie si parametrov spojenia (ako napr. 3 handshake),
- umožniť poslať dáta po fragmentoch s veľkosťou zadanou zo vstupu od používateľa,
- umožňovať overenie integrity poslanej správy na strane príjemcu,
- vedieť znova poslať stratenú alebo poškodenú správu,
- zabezpečenie vzájomnej kontroly medzi uzlami, či je účastník na druhej strane spojenia stále aktívny,
- umožniť úmyselne vytvoriť chybu v niektorej z prenášaných správ s cieľom simulovať poškodené dáta,
- ostatné veci, ktoré považujem za dôležité.

Na splnenie týchto kritérií boli použité metódy, ako napríklad:

- Metóda CRC16 — na kontrolu integrity prenesenej správy.
- Metóda Selective Repeat (SR) so Sliding window — na zabezpečenie spoľahlivého prenosu dát.
- Metóda Keep-Alive — na udržanie aktívneho spojenia.

Krátky opis dokumentácii po častiach:

1. aké hlavičky protokolu som zvolil a prečo
2. ako funguje CRC, ako ho vypočítame, aké polynómy existujú pre neho

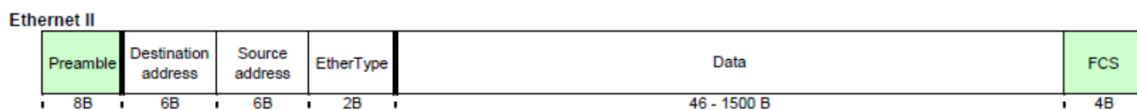
V prvej časti je opísané, aké hlavičky protokolu som zvolil a prečo. V druhej časti je opísané, ako funguje CRC, ako ho vypočítame, aké polynómy existujú pre neho a ako môže vyzeráť algoritmus. Taktiež je opísané aký a prečo som zvolil CRC. V nasledujúcich častiach budú opísané metódy na zabezpečenie spoľahlivého prenosu dát, vysvetlený pojem „Window“ a iné, a na čo oni slúžia a ako

ich použijeme, opísaná metóda SR s príkladom. V predposlednej časti bude opísaná metóda Keep-Alive, jej parametre, nastavenie, udržanie a ukončenie pripojenia.

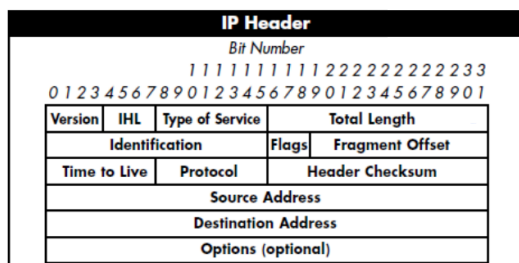
Taktiež, na lepšie pochopenie fungovania vytvoreného prototypu, v záverečnej časti boli vytvorené diagramy opisujúce predpokladané správanie uzla/uzlov.

Štruktúra hlavičiek protokolu

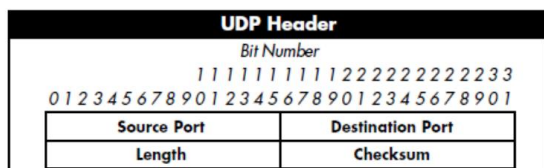
1.1 Opis hlavičky v bajtovej reprezentácii



- Minimálny počet dáta pre Ethernet II je 46B, maximálny - 1500B.
- IP hlavička nám obsahuje ešte 20-60(s opciami) bajt:



- Nasledovne v transportnej vrstve nám pribúda UDP ktorý ma 8 bajt.



Spolu je to 1432-1472 bajt pre UDP dáta.

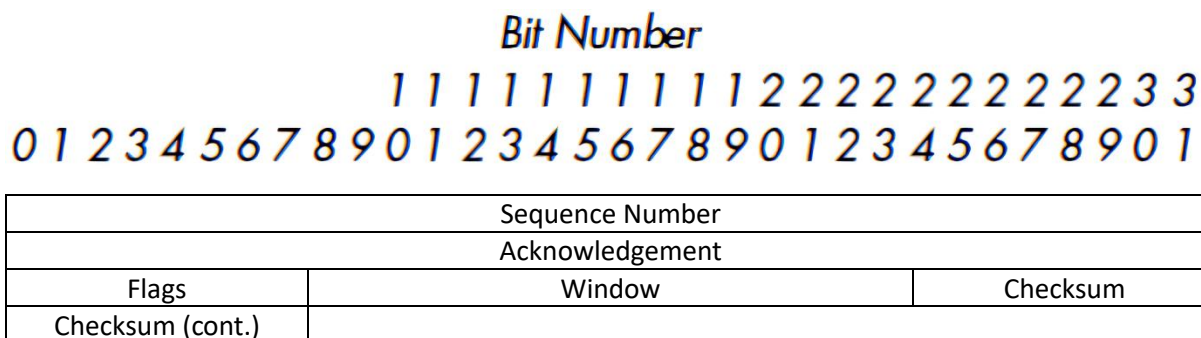
Pre náš protokol sme použijeme:

- Sequence Number – 4 bajt
- Acknowledgement – 4 bajt
- Flags – 1 bajt
- Window – 2 bajt
- Checksum – 2 bajt

Spolu: 13 bajtov

Pre kontrolu segmentácii pre svoj protokol a UDP, potrebujem nastaviť maximálny počet bajtov pre položku dáta — 1459, považujem, že IP hlavička nebude mať opcii. Ako začiatočnú hodnotu zvolím si 1419B. Bude to rozmer jedného segmentu dát v nasej hlavičke protokolu. Počas behu programu tuto hodnotu je možné meniť dynamické pred a po poslaní správy alebo súboru.

1.2 Štruktúra hlavičky vlastného protokolu



1.3 Opis použitých polí v hlavičke.

- Checksum používaná na overenie integrity prenášaných dát. Kontroluje, či neboli dáta počas prenosu poškodené. V tejto časti použijeme algoritmus na kontrolu integrity prenesenej správy - CRC16.
- Sequence Number je určený pre každý odoslaný fragment, aby nestratil poradie, v ktorom boli dáta odoslané. SN obsahuje poradie odoslaného fragmentu. A slúžia na indikáciu prijatých a stratených fragmentov, čo umožňuje zase odoslať segmenty k príjemcovi. Príjemca, keď prime fragment vie na aký fragment musí poslať potvrdenie ACK.
- Acknowledgement slúži na potvrdenie prijatia fragmentov, odosiela sekvenčné číslo prijatého segmentu.
- Window má určitú dĺžku, koľko dát (v počte segmentov) môžeme poslať bez čakania na potvrdenie. Keď dostaneme spätnú väzbu, že segment bol prijatý, tak posunieme okno na sledujúci fragment. Keď počet odoslaných fragmentov bez podvedenia nemôže presiahnuť rozmer okna, tak musíme čakať na aspoň jeden potvrdenie, po ktorom môžeme poslať nasledujúci fragment. Keď potvrdenie nepríde, musíme odosielanie fragmentu opakovať, na to je vhodné použiť timeout, po ktorom kontrolujeme, ktoré fragmenty nepošli. To umožňuje nám aby sieťová linka nebola preťažaná.
- Flags – nastavenie príznakov.

V políčko Flags ma nasledujúce príznaky:

0b00000000

0. Prenos správy
1. Prenos súboru
2. Udržanie spojenia, Keep-Alive
3. ACK(potvrdenie)
4. Indikácia prvého segmentu
5. Indikácia posledného fragmentu
6. SYN(použitý pri inicializácii spojenia)

7. *FIN(príznak, ktorý slúži na ukončenie spojenia)*

Pre NACK správy bolo použitá sekvencia 0b00000000, a pre potvrdenie prijatia súboru 0b11111111.

Metód na kontrolu integrity prenesenej správy

2.1 Úvod do metódy kontrolu integrity prenesenej správy

Checksum slúži na kontrolu prenesených správ, deteguje chyby, čo umožňuje poslať signál odosielateľovi oznamujúci, že prešiel poškodený segment.

Na kontrolu integrity existuje veľa možných metód takých ako: Internet Checksum (TCP/UDP Checksum), CRC (Cyclic Redundancy Check), VRC, LRC, Hamming, Reed-Solomon, HMAC (Hash-based Message Authentication Code), MD5/SHA-1/SHA-256 a iné.

Bol zvolený CRC16, je to dosť bezpečný a spoľahlivý algoritmus na kontrolu správnosti prenosných dát v rámci požadovanej úlohy, dosť rýchly v porovnaní s inými metódami, nepotrebuje veľkej výpočtovej zložitosti.

2.2 Metóda na tvorbu chýb počas prenosu dát

Pre tvorbu chýb pre správy/súbore je potrebné pre odosielaním správy/súboru napísať `reverse_data` po ktorej nasleduje text/názov súboru. Tá položka zavolá metódu, ktorá pred poslaním správy urobí chyby (s nastavenou hodnotou, percentom chýb, aký používateľ môže upraviť manuálne), tak že spraví CRC pre segment, ktorý bude v obrátenom tvare, keď správa je z jedného symbolu, tak posune znak o jeden.

Poškodené segmenty ďalej budú prijaté a bude odoslaný NACK pre zopakované odoslanie poškodeného fragmentu.

2.3 Ako funguje CRC16?

CRC16 používa na výpočet kontrolného súčtu polynómy. Polynómy vyzerajú nasledovne:

$$f(x) = a_n x^n + a_{n-1} x^{n-1} + \dots + a_1 x + a_0$$

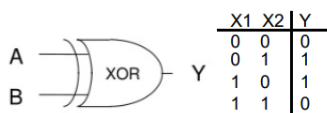
CRC16 používa 16-bitový polynóm, čo znamená, že výsledkom výpočtu bude vždy 16-bitové číslo.

Na strane odosielateľa z dát ktoré chceme odoslať sa vypočíta Checksum, polinomialna funkcia slúži pre výpočet CRC. Potom odsiaľame dáta s sumou, prijímateľ musí overiť správnosť prenášaných dát, preto použije funkciu na overenie, keď pri výpočte, ako výsledok bude mať n núl, tak to znamená, že dáta nie sú poškodené. Počet núl závisí od zvolenej polinomialnej funkcie. Zvolená funkcia je tiež veľmi dôležitou.

2.4 Výpočet CRC16

Takto vyzerá XOR funkcia, ktorú sme používame pri výpočte CRC16:

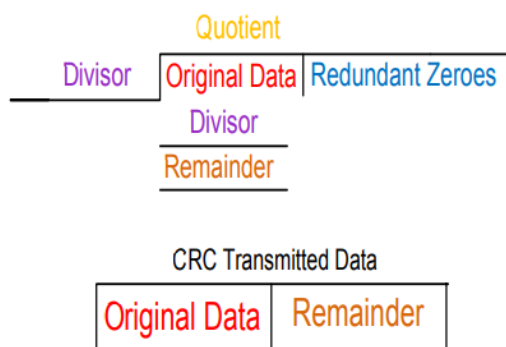
XOR-Function



Nasledujúci príklad ukazuje, ako odosielateľ vypočíta CRC pred odosielaním správy. Zvolím si ľahší polynóm, aby ukázať ako funguje výpočty pre CRC.

Mame správu, ktorú chceme odoslať v bitovej reprezentácii. Mame polynóm: $x^2 + 1$, aký zapíšeme ako 101. Polynóm môžeme zobrazit ako $x^0 + x^1 \dots x^n$, kde n v našom prípade bude 2(pre CRC16, n = 15). Ak nejaký prvok z postupnosti chyba, dáme namiesto neho 0, inak 1.

- Koeficient pri x^2 je 1,
- Koeficient pri x^1 je 0,
- Koeficient pri x^0 (konštanta) je 1.

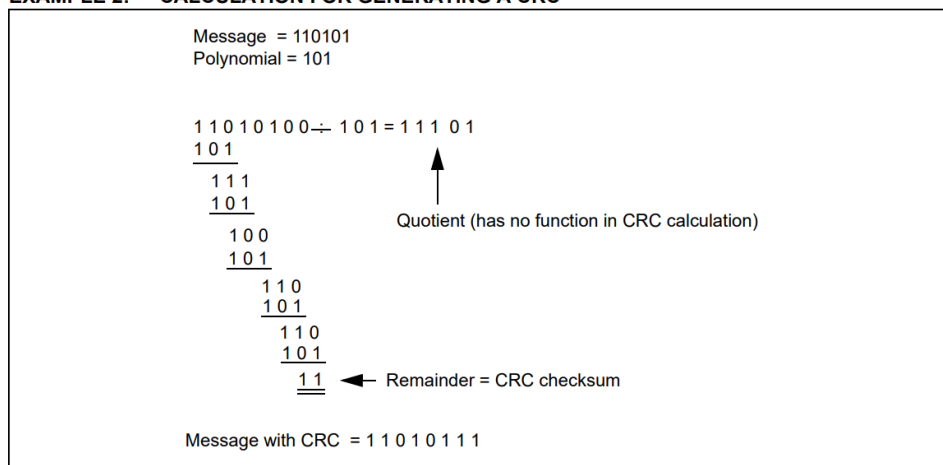


(7)

Fig: Procedure for CRC Encoder

Prepíšem správu v bytovom tvare a prepíšeme dĺžku polynómu – 1 núl k správe. Začnem “XORovať” pod prvými byt napíšem náš polynóm(alebo nuly namiesto polynómu, ak budeme naliavši bit 0) a spravím XOR, zápismi quotient 1(alebo 0). Dáme dole jeden bit a cyklus zopakujem, kým nebudem môcť pokračovať(nebudem môcť dať dole naprávši bit). Zostane nám nakoniec CRC(dĺžku polynómu – 1).

EXAMPLE 2: CALCULATION FOR GENERATING A CRC



(4)

Keď dáta prídu na ku príjemcu musíme skontrolovať ich, urobíme tak, že zopakujeme tieto operácie nad dátami s kontrolnej sumou.

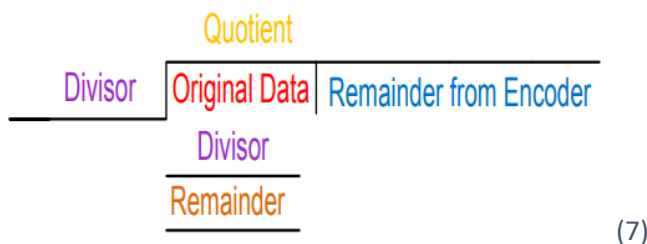
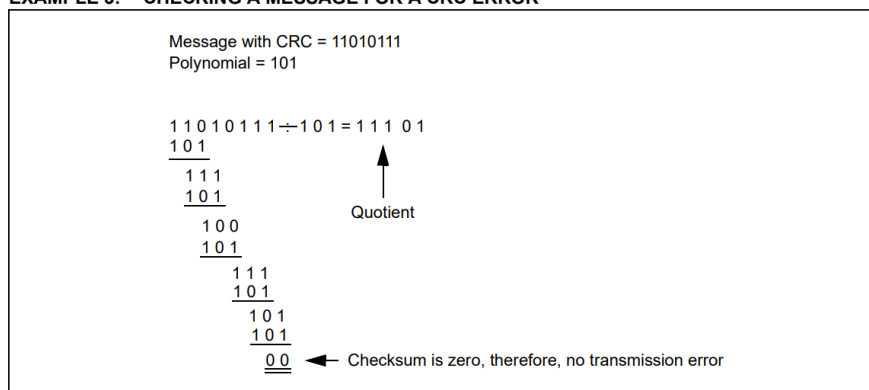


Fig: Procedure for CRC Decoder

EXAMPLE 3: CHECKING A MESSAGE FOR A CRC ERROR



V tomto príklade po všetkých výpočtoch nám vyšiel výsledok 0, čo znamená ,že dáta nie sú poškodené.

2.5Zvolený CRC (CRC-16-CCITT) a implementácia

Je veľa možností zvoliť si polynómialnu funkciu. Ale jednou z najpopulárnejších je polynóm $x^{16} + x^{12} + x^5 + 1$ (CRC-16-CCITT). Myslím bude to dobre riešenie pre daný typ úloh, preto ho implementoval vo svojej prave.

Na implementáciu zvoleného algoritmu použime knižnicu pre CRC. Tato knižnica obsahuje crc16, kde je možné nastaviť polynóm, začiatočnú hodnotu a XOR.

```
from crcmod import crcmod
...
def calculate_checksum(self, data):
    crc16_ccitt = crcmod.mkCrcFun(0x11021, initCrc=0xFFFF, xorOut=0x0000)
    if isinstance(data, str):
        data = data.encode('utf-8')
    crc = crc16_ccitt(data)
    return crc
```

Metóda na zabezpečenie spoľahlivého prenosu dát (ARQ)

3.1Metódy na zabezpečenie spoľahlivého prenosu dát

Na zabezpečenie spoľahlivého prenosu dát existuje rôzne metódy, ako napríklad:

- Modifikácie GBN/SR
- SR (Selective Repeat) odošle niekoľko segmentov za sebou, a ak nejaký segment bude poškodený alebo sa stratí odošle iba tie segmenty, ktoré chýbajú.

- GBN (Go-Back-N) odošle niekoľko segmentov za sebou, a ak nejaký segment bude poškodený alebo sa stratí bude musieť odoslať všetky segmenty od neho.
- S&W (Stop-and-Wait) odošle jeden segment alebo a čaká na potvrdenie. Nie je efektívny, t. j. musí pri každom odoslaní čakať na potvrdenie.
- Iné

Na zabezpečenie spoľahlivého prenosu dát je neimplementované použitie modifikácii SR(Selective Repeat) so Sliding Window.

3.2SR metóda

SR – metóda na opakované odosielanie iba tých segmentov, ktoré boli poškodené alebo stratené počas prenosu. Je to efektívna metóda, ktorá znižuje zbytočné odosielanie rámcov, ktorý neboli poškodené. Odosiela NACK pre balíky, aký je stratené alebo poškodené.

Pre tú metódu je vhodné použiť v hlavičke protokolu Sequence Number, Acknowledgement(ACK), Window. Window slúži na udržanie maximálneho počtu balíkov, ktoré môžu byť odoslané súčasne bez potvrdenia. Keď odosieme fragmenty pozeráme na veľkosť okna, keď počet fragmentov bez podvedenia je rovnaký ako okno, tak cvákame v metóde `wait_for_acks()`, keď prídu všetky ACK podvedenia a pokračujem prenos, keď podvedenia neprídu za určitý čas, tak prenos zopakujeme pre fragmenty pre ktoré neprišiel príznak ACK. Pre testovanie timeout a hľadanie informácie prišiel k tomu, že timeout je vhodné nastaviť ako $1 + (window - 1) * 0.2$. S touto hodnotou nebudem mať veľa problémov, ako pri konštantnej hodnote. Keď bola linka veľmi preťažená, opakované posielanie fragmentov bolo zbytočné a veľmi pomalé.

```
while True:
    if seq_num < self.base + self.window_size:
        self.send_file_fragment(file_seg, seq_num, checksum, flags)
        break
    else:
        if not self.wait_for_acks():
            print("Timeout expired. Resending fragments.")
            self.check_timeouts()

def check_timeouts(self):
    for seq_num in range(self.base, self.seq_num):
        if seq_num not in self.ack_received:
            data = self.retrieve_data_for_seq_num(seq_num)
            if data is not None:
                self.checksum = self.calculate_checksum(data)
                self.seq_num = seq_num
                self.send_fragment(data, self.flags)
```

3.3Výber veľkosti okna (N)

Treba si vedieť aký rozmer okna treba si zvoliť pre efektívnejší prenos dát. Pre túto úlohu bola neimplementovaná možnosť meniť počiatočný rozmer okna počas behu programu. Začiatočnú hodnotu okna v konfigurácii zvolil ako 10. Túto hodnotu je možné uložiť v konfigurácii.

3.4Príklad okna



3.5Efektívnosť algoritmu SR

10

Metóda na udržanie spojenia – Keep-Alive

4.1 Úvod do metódy Keep-Alive

Metóda na udržanie spojenia Keep-Alive umožňuje efektívne používať spojenie bez potreby opätovného vytvárania spojenia, čo umožňuje ovplyvniť na efektívne používanie času a zdrojov.

4.2 Ako funguje metóda na udržanie spojenia, a jej parametre

Máme nastavené 3 parametre pre fungovanie keep-live metódy:

- `keepalive_interval = 5`
`keepalive_probes = 3`

1. **Keepalive interval** – interval medzi po sebe idúcimi keepalive probe.
2. **Keep-Alive Probes** – počet pokusov keep-alive, po ktorých môžeme povedať, že sa pripojenie zlyhalo.

Ak počas **Keep-Alive Time** odosielateľ a príjemca nekontaktujú (prenos sprav/súboru), protokol začína odosielať keep-alive správy. Je to prázdne pakety, ktorý neobsahujú dáta, ale obsahujú nastavený príznak Keep-Alive.

Keď dvaja používatelia prestanú byť aktívne (čas spočítame od poslednej prijatej správy), ony vzájomne odosielať a prijímajú pakety s príznakom Keep-Alive pre potvrdenie, že pripojenie je aktívne. Tým vedia, že pripojenie. Ak sprava s príznakom nepríde v čase **Keepalive interval**, tak jedna strana bude pokračovať posilať keep-alive pakety, kým počet pokusov **Keep-Alive Probes** nebude dosiahnuto. Potom môžeme povedať, že sa pripojenie zlyhalo a zatvoriť pripojenie.

4.3 Ako sa nastavuje pripojenie.

Pripojenie nastavujem keď používateľ chce poslať prvú správu alebo dáta. Používam 3-way handshake.

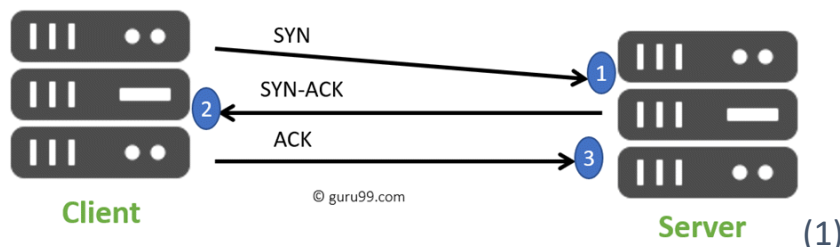
Môj protokol, ktorý ma príznaky ACK, SYN.

Máme štyri typy správ: Fin, ACK, Syn a SYN-ACK. SYN-ACK je stav, keď sú nastavené príznaky SYN a ACK.

No.	Time	Source	Destination	Protocol	Length	Info
216	330.998314	10.10...	10.10.0.2	MyProtocol	55	55001 → 55002 Len=13
217	331.001043	10.10...	10.10.0.3	MyProtocol	60	55002 → 55001 Len=13
218	331.001427	10.10...	10.10.0.2	MyProtocol	55	55001 → 55002 Len=13

> Frame 217: 60 bytes on wire (480 bits), 60 bytes captured (480 bits) on interface
> Ethernet II, Src: ASUSTekCOMPU_4c:13:09 (e8:9c:25:4c:13:09), Dst: ASUSTekCOMPU_4c:
> Internet Protocol Version 4, Src: 10.10.0.2, Dst: 10.10.0.3
> User Datagram Protocol, Src Port: 55002, Dst Port: 55001
▼ My Application Transmission Data Protocol
Sequence Number: 13387
Acknowledgement: 17510
▼ Flags: 0x48
0... .. = MES (Message): Not set
.0.. .. = FIL (File): Not set
..0. = KPL (Keep-alive): Not set
...1 = ACK (Acknowledgement): Set
.... 0... = FF (FIRST_Fragment): Not set
.... .0.. = LF (Last_Fragment): Not set
.... ..1. = SYN (Synchronize): Set
.... ...0 = FIN (Finish): Not set
Window Size: 10
Checksum: 0x0000

- Fin – používa sa na ukončenie spojenia
- Syn – Používa sa na iniciovanie a nadviazanie spojenia.
- ACK – Pomáha potvrdiť druhej strane, že dostala SYN.
- Syn-ACK – potvrdí prijatie ACK a pošle svoju žiadosť o synchronizáciu SYN.



Keď prvá strana odosiela segment so SYN, chce navádzať spojenie so druhým používateľom. Ďalej server odpovie SYN-ACK signálom na požiadavku o nadviazanie spojenia. Keď používame P2P komunikácii, tak jedna strana je aj klientom aj serverom, pre lepšie pochopenie boli zobrazené dve strany: Klient, čo chce navádzať spojenie a server, ktorý prime požiadavku o spojenie a vzájomný prenos dát.

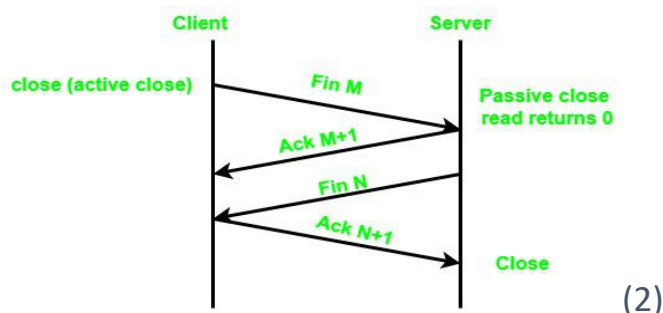
Klientovi zostalo iba potvrdiť žiadosť poslaním potvrdenia ACK. Na tomto momente klient a server vytvorili stabilné spojenie a môžu obojstranné posilať dáta jeden druhému.

Nasledujúci príklad z reálnej situácie, ktorá môže nastať. Zvyčajne syn je náhodne číslo z intervalu(napr.0 – 2^{16} , nastavil som taky interval, aby bolo prehľadnejšie sledovať), a ACK je číslo segmentu, ktorý je prijatý.

Sequence Number: 17509 Acknowledgement: 0 Flags: 0x40	Sequence Number: 13387 Acknowledgement: 17510 Flags: 0x48	Sequence Number: 17510 Acknowledgement: 13388 Flags: 0x08
0... .. = MES (Message): Not set .0.. .. = FIL (File): Not set ..0. = KPL (Keep-alive): Not set ...0 = ACK (Acknowledgement): Not set 0... = FF (FIRST_Fragment): Not set 0... = LF (Last_Fragment): Not set 1. = SYN (Synchronize): Set 0... = FIN (Finish): Not set	0... .. = MES (Message): Not set .0.. .. = FIL (File): Not set ..0. = KPL (Keep-alive): Not set ...1 = ACK (Acknowledgement): Set 0... = FF (FIRST_Fragment): Not set 0... = LF (Last_Fragment): Not set 1. = SYN (Synchronize): Set 0... = FIN (Finish): Not set	0... .. = MES (Message): Not set .0.. .. = FIL (File): Not set ..0. = KPL (Keep-alive): Not set ...1 = ACK (Acknowledgement): Set 0... = FF (FIRST_Fragment): Not set 0... = LF (Last_Fragment): Not set 0... = SYN (Synchronize): Not set 0... = FIN (Finish): Not set

4.4 Ako sa ukončuje pripojenie.

Pre správnu komunikáciu siete musíme vedieť poslať aj signál na ukončenie spojenia. Je to podobne, ako aj pri 3-way-handshake, pre navádzanie spojenia.



V tomto prípade, ak klient chce ukončiť pripojenie, musí poslať FIN správu (príznak FIN = 1). Server potvrdí túto požiadavku a pošle ACK správu(príznak ACK = 1). Týmto klient ukončí spojenie na svojej strane, ale server tiež musí ukončiť spojenie, preto server odosiela FIN správu, klient ju prijme a odošle potvrdenie ACK pre ukončenie. Server prime ACK a spojenie môžeme považovať, ako ukončené.

Popis časti programu

4.1 Zatáčok programu

Použite knižnice:

```
import math
import os
import random
import socket
import struct
import threading
import time

from colorama import Fore, Style, init
import configparser
from crcmod import crcmod
```

- Colorama – pre farebný výpis
- Os – uloženie cesty k súboru
- Configparser – pre konfiguračný súbor
- Crcmod – pre výpočet CRC16

Na začiatku program načíta konfiguračný súbor, ak jestvuje, inak načíta default hodnoty a nastavenia. Následne vypíše informácie, ako začať komunikáciu, zoznam použiteľných príkazov a najdôležitejšie konfiguračne nastavenia, ako napríklad IP adresy a porty pre UDP komunikáciu.

```
How to start:
1) Set connection configuration (CSI, CSP, CCP)
2) Use commands to send messages or files, or change configuration
3) Close connection (CSI, CSP, CCP)

Available commands:
0) CCR - create socket, bind and receive message | after configure connection (CSI, CSP, CCP)

1)
  help - Show commands
  ipconfig - show ip configuration
  add_cmds - show additional commands (show_keepalive_activity, show_send_info, show_receive_info, ...)

2)
  CSI <new_ip> - Change LOCAL_IP, CRI <new_ip> - Change REMOTE_IP
  CSP <new_port> - Change OUT_PORT, CCP <new_port> - Change IN_PORT

3)
  change_fragment_size <fragment_size> - change fragment size
  change_storage_path <new_path> - change storage path
  change_error_rate <error_rate> - change error rate
  change_window_size <window_size> - change window size
  change_check_ack_interval <check_ack_interval> - change check ack interval

4)
  send_file - send file to server, if start with reverse_data|<file_name> -> reverse some segment
  send_message - send message to server (stop - stop input messages),
  if start with reverse_data|<data> -> reverse data

5)
  save_config - save configuration
  load_default_config - load default configuration
```

```
6)      auto_bind - auto bind socket

7)      close_connection - close the connection
        exit - exit the program

Connection Configuration:
-----
LOCAL_IP: 10.10.0.3
REMOTE_IP: 10.10.0.2
OUT_PORT: 55002
IN_PORT: 55001
FRAGMENT_SIZE: 1370
error_rate: 0.04
window_size: 10
check_ack_interval: 0.01
STORAGE_PATH: files/
show_keepalive_activity: False
show_send_info: False
show_receive_info: False
-----

~~~Auto binding socket~~~

Socket created
Socket bind complete on 10.10.0.3:55001
Message receive start
```

4.2 Konfigurácia v súbore

Konfigurácia v súbore vyzerá nasledovne:

```
[Settings]
show_keepalive_activity_control = False
show_sequence_number_control = False
show_receive_control = False
show_crc_check_control = False
local_ip = 10.10.0.3
remote_ip = 10.10.0.2
out_port = 55002
in_port = 55001
fragment_size = 1370
error_rate = 0.04
storage_path = files/
keepalive_interval = 5
keepalive_probes = 3
window_size = 10
check_ack_interval = 0.01
auto_bind = True
```

Prvých štyri nastavenia je výpis textu do konzoly, čo je default. Ďalej sú IP adresy a porty, veľkosť fragmentu, počet chýb, miesto uloženia súboru, nastavenie Keep-alive, veľkosť okna a interval kontroly ACK.

auto_bind = True, je ukážka pri štarte programu, že porty a adresy nastavené správne a môžeme začínať.

Funkcia na uloženie konfigurácie:

```
def save_config():
    global show_keepalive_activity_control, show_sequence_number_control, show_receive_control,
    LOCAL_IP, OUT_PORT, \
        IN_PORT, FRAGMENT_SIZE, error_rate, STORAGE_PATH, keepalive_interval, keepalive_probes,
    auto_bind, \
        show_crc_check_control

    config['Settings'] = {
        'show_keepalive_activity_control': str(show_keepalive_activity_control),
        'show_sequence_number_control': str(show_sequence_number_control),
        'show_receive_control': str(show_receive_control),
        'show_crc_check_control': str(show_crc_check_control),
        'LOCAL_IP': LOCAL_IP,
        'REMOTE_IP': REMOTE_IP,
        'OUT_PORT': str(OUT_PORT),
        'IN_PORT': str(IN_PORT),
        'FRAGMENT_SIZE': str(FRAGMENT_SIZE),
        'error_rate': str(error_rate),
        'STORAGE_PATH': STORAGE_PATH,
        'keepalive_interval': str(keepalive_interval),
        'keepalive_probes': str(keepalive_probes),
        'window_size': str(window_size),
        'check_ack_interval': str(check_ack_interval),
        'auto_bind': str(auto_bind)
    }
```

4.2 Iné vlastnosti programu

Veľkosť fragmentu je možné meniť pred poslaním správy:

```
Fragment size changed to 6
send_message
Enter the message to send or stop to stop sanding message:
Hello Bob
Text size: 9 bytes
Count of fragments: 2
Fragment size: 6 bytes
Last fragment size: 3 bytes
All fragments received.
Received last fragment 17682.
All fragments received, message:
Hello Bob
Received 2 fragments.
Transition time: 0.502129077911377s.
End message transmission
```

Výpis po prijatie súboru:

```
Transition time: 1.276594638824463s.
File name: katalog-predmetov_2024-25.pdf
File path: files/katalog-predmetov_2024-25.pdf
File saved to: C:\Users\dokan\Desktop\Protocol\0dovzdanie\files\katalog-predmetov_2024-25.pdf
Total file size: 930339 bytes
Total fragments received: 640 (including file name)
Last fragment size: 773 bytes
First fragment is file name, size 29 bytes.
638 fragments with size 1457 bytes.
End message transmission
```

Hlavná trieda programu:

```
class Protocol:
    def __init__(self, seq_num, ack_num, flags, window_size, data):
```

```
self.seq_num = int(seq_num)
self.ack_num = int(ack_num)
self.flags = int(flags)
self.window_size = int(window_size)
self.checksum = 0
self.data = ""

self.last_received_time = time.time()

self.wrong_data = False
self.sent_packets = {}
self.received_packets = {}

self.file_path = None

self.base = 0
self.ack_received = set()

self.timeout = 1 + (self.window_size - 1) * 0.2
self.received_seq_numbers = set()
```

Hlavný cyklus:

```
while True:
    input_command = input()
    commands_to_execute(input_command)

    if input_command == "exit":
        exit(0)
```

Pri odosielaní súboru odosielam ako prvý fragment názov súboru. Následne odosielam fragmenty súboru s využitím metódy SR a mechanizmu sliding window. Súbor spracovávam postupne, pričom priebežne uvoľňujem pamäť pre zoznam fragmentov.

```
def save_part_of_file(self):
    os.makedirs('files', exist_ok=True)

    with open(self.file_path, 'ab') as f:
        sorted_keys = sorted(self.received_packets.keys())
        first_seq_num = min(sorted_keys)
        half_index = len(sorted_keys) // 4
        for seq_num in sorted_keys[:half_index]:
            if seq_num == first_seq_num: # Skip the first fragment (file name)
                continue
            f.write(self.received_packets[seq_num])

def clear_used_fragments(self):
    sorted_keys = sorted(self.received_packets.keys())
    first_seq_num = min(sorted_keys)
    half_index = len(sorted_keys) // 4
    for seq_num in sorted_keys[:half_index]:
        if seq_num == first_seq_num: # Skip the first fragment (file name)
            continue
        del self.received_packets[seq_num]
```


Testovacie príklady so Wireshark

Prenos bez chyb:

No.	Time	Source	Destination	Protocol	Length	Info
1708	7288.769350	10.10...	10.10.0.2	MyProtocol	55	55001 → 55002 Len=13
1709	7288.769401	10.10...	10.10.0.3	MyProtocol	1512	55002 → 55001 Len=1470
1710	7288.769401	10.10...	10.10.0.3	MyProtocol	1512	55002 → 55001 Len=1470
1711	7288.769744	10.10...	10.10.0.2	MyProtocol	55	55001 → 55002 Len=13
1712	7288.770128	10.10...	10.10.0.2	MyProtocol	55	55001 → 55002 Len=13
1713	7288.770175	10.10...	10.10.0.3	MyProtocol	1512	55002 → 55001 Len=1470
1714	7288.770175	10.10...	10.10.0.3	MyProtocol	1512	55002 → 55001 Len=1470
1715	7288.770175	10.10...	10.10.0.3	MyProtocol	1512	55002 → 55001 Len=1470
1716	7288.770509	10.10...	10.10.0.2	MyProtocol	55	55001 → 55002 Len=13
1717	7288.770839	10.10...	10.10.0.3	MyProtocol	1512	55002 → 55001 Len=1470
1718	7288.770905	10.10...	10.10.0.2	MyProtocol	55	55001 → 55002 Len=13
1719	7288.771287	10.10...	10.10.0.2	MyProtocol	55	55001 → 55002 Len=13
1720	7288.771498	10.10...	10.10.0.3	MyProtocol	1512	55002 → 55001 Len=1470
1721	7288.771666	10.10...	10.10.0.2	MyProtocol	55	55001 → 55002 Len=13
1722	7288.772154	10.10...	10.10.0.3	MyProtocol	1512	55002 → 55001 Len=1470
1723	7288.772154	10.10...	10.10.0.3	MyProtocol	1512	55002 → 55001 Len=1470
1724	7288.772803	10.10...	10.10.0.3	MyProtocol	1512	55002 → 55001 Len=1470

> Frame 1713: 1512 bytes on wire (12096 bits), 1512 bytes captured (12096 bits) on interface
 > Ethernet II, Src: ASUSTekCOMPU_4c:13:09 (e8:9c:25:4c:13:09), Dst: ASUSTekCOMPU_4c:16:9c:
 > Internet Protocol Version 4, Src: 10.10.0.2, Dst: 10.10.0.3
 > User Datagram Protocol, Src Port: 55002, Dst Port: 55001
 ✓ My Application Transmission Data Protocol
 Sequence Number: 57507
 Acknowledgement: 0
 Flags: 0x02
 0... = MES (Message): Not set
 .1.. = FIL (File): Set
 ..0. = KPL (Keep-alive): Not set
 ...0 = ACK (Acknowledgement): Not set
 0... = FF (FIRST_Fragment): Not set
 0.. = LF (Last_Fragment): Not set
 0. = SYN (Synchronize): Not set
 0 = FIN (Finish): Not set
 Window Size: 10
 Checksum: 0x3c4a
 Data: 1457 bytes

Posledný fragment:

2739	7289.294728	10.10...	10.10.0.3	MyProtocol	1512	55002 → 55001 Len=1470
2740	7289.294744	10.10...	10.10.0.3	MyProtocol	1512	55002 → 55001 Len=1470
2741	7289.295131	10.10...	10.10.0.3	MyProtocol	828	55002 → 55001 Len=786

Kontrolná sprava:

2746	7289.305754	10.10... 10.10.0.3	MyProtocol	60	55002 → 55001	Len=13
2747	7300.014710	10.10... 10.10.0.3	MyProtocol	60	55001 → 55002	Len=13

```
> Frame 2746: 60 bytes on wire (480 bits), 60 bytes captured (480 bits) on interface
> Ethernet II, Src: ASUSTekCOMPU_4c:13:09 (e8:9c:25:4c:13:09), Dst: ASUSTekCOMPU_4c:
> Internet Protocol Version 4, Src: 10.10.0.2, Dst: 10.10.0.3
> User Datagram Protocol, Src Port: 55002, Dst Port: 55001
▼ My Application Transmission Data Protocol
  Sequence Number: 0
  Acknowledgement: 0
  Flags: 0xff
    1... .... = MES (Message): Set
    .1.. .... = FIL (File): Set
    ..1. .... = KPL (Keep-alive): Set
    ...1 .... = ACK (Acknowledgement): Set
    .... 1... = FF (FIRST_Fragment): Set
    .... .1.. = LF (Last_Fragment): Set
    .... ..1. = SYN (Synchronize): Set
    .... ...1 = FIN (Finish): Set
  Window Size: 10
  Checksum: 0x0000
```

Prenos s chybami:

```
Data is encoded wrong in fragment 17263, seq num 17263.
Missing data for seq num 17263

Resending missing data for seq num 17263.
```

(Bol odoslaný poškodený fragment, potom dostali

sme NACK a zase odoslali fragment)

14	11.394834	10.10... 10.10.0.2	MyProtocol	311	55001 → 55002	Len=269
15	11.395138	10.10... 10.10.0.2	MyProtocol	311	55001 → 55002	Len=269
16	11.395438	10.10... 10.10.0.2	MyProtocol	311	55001 → 55002	Len=269

```
> Frame 14: 311 bytes on wire (2488 bits), 311 bytes captured (2488 bits) on interface \D
> Ethernet II, Src: ASUSTekCOMPU_4c:16:9c (e8:9c:25:4c:16:9c), Dst: ASUSTekCOMPU_4c:13:09
> Internet Protocol Version 4, Src: 10.10.0.3, Dst: 10.10.0.2
> User Datagram Protocol, Src Port: 55001, Dst Port: 55002
▼ My Application Transmission Data Protocol
  Sequence Number: 17263
  Acknowledgement: 0
  Flags: 0x02
    0... .... = MES (Message): Not set
    .1.. .... = FIL (File): Set
    ..0. .... = KPL (Keep-alive): Not set
    ...0 .... = ACK (Acknowledgement): Not set
    .... 0... = FF (FIRST_Fragment): Not set
    .... .0.. = LF (Last_Fragment): Not set
    .... ..0. = SYN (Synchronize): Not set
    .... ...0 = FIN (Finish): Not set
  Window Size: 10
  Checksum: 0x3874
```

NACK sprava

23	11.398498	10.10... 10.10.0.3	MyProtocol	60	55002 → 55001	Len=13
24	11.398498	10.10... 10.10.0.3	MyProtocol	60	55002 → 55001	Len=13
25	11.398498	10.10... 10.10.0.3	MyProtocol	60	55002 → 55001	Len=13

```
Acknowledgement: 17263
▼ Flags: 0x00
  0... .... = MES (Message): Not set
  .0.. .... = FIL (File): Not set
  ..0. .... = KPL (Keep-alive): Not set
  ...0 .... = ACK (Acknowledgement): Not set
  .... 0... = FF (FIRST_Fragment): Not set
  .... .0.. = LF (Last_Fragment): Not set
  .... ..0. = SYN (Synchronize): Not set
  .... ...0 = FIN (Finish): Not set
Window Size: 10
```

Správne odosielanie:

26	11.398938	10.10.10.10.0.2	MyProtocol	311	55001 → 55002	Len=269
27	11.399421	10.10.10.10.0.3	MyProtocol	60	55002 → 55001	Len=13

> Frame 26: 311 bytes on wire (2488 bits), 311 bytes captured (2488 bits) on interface \Device\NPF{...} Ethernet II, Src: ASUSTekCOMPU_4c:16:9c (e8:9c:25:4c:16:9c), Dst: ASUSTekCOMPU_4c:13:09 (08:00:00:00:00:00)	0000 e8 9c 25 4c 13 09 e8 9c 25 4c 16 9c 08 00 45 00 ..%L....%L....E..
> Internet Protocol Version 4, Src: 10.10.0.3, Dst: 10.10.0.2	0010 01 29 d7 d3 00 00 80 11 00 00 0a 0a 00 03 0a 0a ..).....P.Cov..
> User Datagram Protocol, Src Port: 55001, Dst Port: 55002	0020 00 02 d6 d9 d6 da 01 15 15 3f 00 00 43 6f 00 00P.Cov..
> My Application Transmission Data Protocol	0030 00 00 00 0a 38 74 61 67 5f 31 2c 20 66 5f 66 ..8ta g_l, f_f
Sequence Number: 17263	0040 6c 61 67 5f 32 2c 20 66 5f 66 6c 61 67 5f 33 2c lag_2, f_flag_3,
Acknowledgement: 0	0050 20 66 5f 66 6c 61 67 5f 34 2c 20 66 5f 66 6c 61 f_flag_4, f fla
Flags: 0x02	0060 67 5f 35 2c 20 66 5f 66 6c 61 67 5f 36 2c 20 66 g_5, f_f lag_6, f
0... = MES (Message): Not set	0070 5f 66 6c 61 67 5f 37 2c 20 66 5f 66 6c 61 67 5f _flag_7, f_flag_
..1... = FIL (File): Set	0080 38 0d 0a 7d 0d 0a 0d 0a 66 75 6e 63 74 69 6f 6e 8...).... function
...0... = KPL (Keep-alive): Not set	0090 20 6d 79 70 72 6f 74 6f 63 6f 6c 2e 64 69 73 73 myproto col.diss
...0... = ACK (Acknowledgement): Not set	00a0 65 63 74 6f 72 28 62 75 66 66 65 72 2c 20 70 69 ector(bu ffer, pi
...0... = FF (FIRST_Fragment): Not set	00b0 6e 66 6f 2c 20 74 72 65 65 29 0d 0a 20 20 20 20 nfo, tre e)..
...0... = LF (Last_Fragment): Not set	00c0 6c 6f 63 61 6c 20 6f 66 66 73 65 74 20 3d 20 30 local of fset = 0
...0... = SYN (Synchronize): Not set	00d0 0d 0a 20 20 20 20 70 69 6e 66 6f 2e 63 6f 6c 73 .. pi nfo.cols
...0... = FIN (Finish): Not set	00e0 2e 70 72 6f 74 6f 63 6f 6c 20 3d 20 22 4d 79 50 .protocol = "MyP
Window Size: 10	00f0 72 6f 74 6f 63 6f 6c 22 0d 0a 20 20 20 20 6c 6f rotocol" .. lo
Checksum: 0x3874	0100 63 61 6c 20 73 75 62 74 72 65 65 20 3d 20 74 72 cal subt ree = tr
Data: 256 bytes	0110 65 65 3a 61 64 64 28 6d 79 70 72 6f 74 6f 63 6f ee:add(m yprotoco
	0120 6c 2c 20 62 75 66 66 65 72 28 29 2c 20 22 4d 79 l, buffe r()), "My
	0130 20 41 70 70 6c 69 63

* Vidíme, že checksum je rovnaký, ale dáta je rôzne, preto na druhej strane pre overenie CRC výsledok nebol správny(0).

Prenos keep-alive:

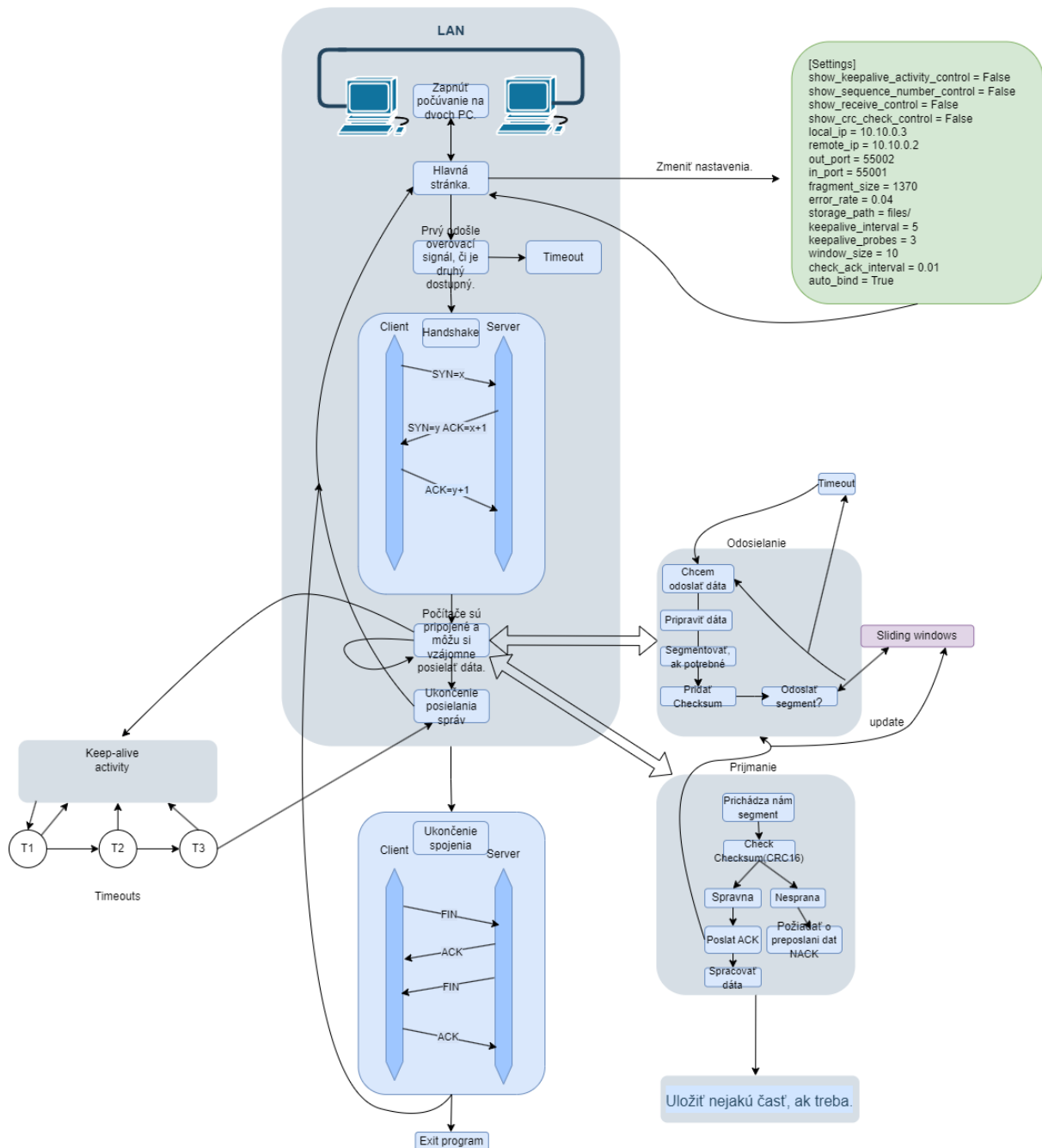
Keep-alive správy majú nastavené vlastné príznaky:

645	1020.007059	10.10.10.10.0.2	MyProtocol	55	55001 → 55002	Len=13
646	1020.879865	10.10.10.10.0.3	MyProtocol	60	55002 → 55001	Len=13
649	1025.087361	10.10.10.10.0.2	MyProtocol	55	55001 → 55002	Len=13
652	1025.879464	10.10.10.10.0.3	MyProtocol	60	55002 → 55001	Len=13
653	1030.087947	10.10.10.10.0.2	MyProtocol	55	55001 → 55002	Len=13
654	1030.880675	10.10.10.10.0.3	MyProtocol	60	55002 → 55001	Len=13
655	1035.088341	10.10.10.10.0.2	MyProtocol	55	55001 → 55002	Len=13

> Frame 649: 55 bytes on wire (440 bits), 55 bytes captured (440 bits) on interface \Device\NPF{...} Ethernet II, Src: ASUSTekCOMPU_4c:16:9c (e8:9c:25:4c:16:9c), Dst: ASUSTekCOMPU_4c:13:09 (08:00:00:00:00:00)	0000 e8 9c 25 4c 13 09 e8 9c 25 4c 16 9c 08 00 45 00 ..%L....%L....E..
> Internet Protocol Version 4, Src: 10.10.0.3, Dst: 10.10.0.2	0010 00 29 d8 a3 00 00 80 11 00 00 0a 0a 00 03 0a 0a ..).....P.Cov..
> User Datagram Protocol, Src Port: 55001, Dst Port: 55002	0020 00 02 d6 d9 d6 da 01 15 15 3f 00 00 43 6f 00 00P.Cov..
> My Application Transmission Data Protocol	0030 00 00 00 0a 0a 00 00 00 00 00 00 00 00 00 00P.Cov..
Sequence Number: 16753	
Acknowledgement: 0	
Flags: 0x04	
0... = MES (Message): Not set	
..0... = FIL (File): Not set	
...1... = KPL (Keep-alive): Set	
...0... = ACK (Acknowledgement): Not set	
...0... = FF (FIRST_Fragment): Not set	
...0... = LF (Last_Fragment): Not set	
...0... = SYN (Synchronize): Not set	
...0... = FIN (Finish): Not set	
Window Size: 10	
Checksum: 0x0000	

Diagramy opisujúce predpokladané správanie uzla/uzlov

Diagramy opisujúce predpokladané správanie uzla/uzlov, použité UML diagramy ako napr. sekvenčný, aktivity a stavový



REFERENCE :

- *Metódy na kontrolu integrity prenesenej správy:*
 - (1) https://en.wikipedia.org/wiki/Cyclic_redundancy_check – Wikipedia, článok o cyklickej redundančnej kontrole (CRC)
 - (2) [CRC Module \(Chapter Excerpt From MSP430x5xx Family, SLAU208\) \(Rev. F\) \(ti.com\)](#) – Texas Instruments dokumentácia pre MSP430 mikrokontroléry. (SLAU398F–August 2012–Revised March 2018). Adresa: Texas Instruments, Post Office Box 655303, Dallas, Texas 75265.
 - (3) [CRC Generating and Checking \(microchip.com\)](#) – Microchip AN730 CRC Generating and Checking
 - (4) <https://ww1.microchip.com/downloads/en/AppNotes/01148a.pdf> – Microchip AN1148 Cyclic Redundancy Check (CRC)
 - (5) [Computation of cyclic redundancy checks - Wikipedia](#) – Wikipedia, článok o výpočtoch CRC.
 - (6) [Implementation of Cyclic Redundancy Check in Data Recovery](#) – Proceedings of the Second International Conference on Electronics and Sustainable Communication Systems (ICESC-2021) IEEE Xplore Part Number: CFP21V66-ART; ISBN: 978-1-6654-2867-5
 - (7) [Microsoft PowerPoint - L09 Timing FPGA v2.pptx \(mit.edu\)](#) – Introductory Digital Systems Laboratory (MIT edu)
- *Metódy na zabezpečenie spoľahlivého prenosu dát:*
 - (1) [Selective Repeat ARQ - Wikipedia](#) – Wikipedia, článok o Selective Repeat (SR) protokole
 - (2) [What is Selective Repeat Protocol? Introduction & Working - Binary Terms](#) – článok o Selective Repeat (SR) protokole
 - (3) <https://www.studytonight.com/post/flow-control-selective-repeat-protocol>
 - (4) [The TCP/IP Guide - TCP Sliding Window Data Transfer and Acknowledgement Mechanics](#)
- *Opis metódy na udržanie spojenia:*
 - (1) [TCP 3-cestné podanie ruky \(SYN, SYN-ACK, ACK\) \(guru99.com\)](#)
 - (2) [Why TCP Connect Termination Need 4-Way-Handshake? - GeeksforGeeks](#)
 - (3) [3 way handshake, TCP Three-way handshake, TCP Synchronization \(omnisecu.com\)](#)
 - (4) <https://www.cspsprotocol.com/tcp-keep-alive/>